Concurrency:

- **`async`**: Marks a function as "asynchronous," meaning it can pause its execution without blocking the thread.
- **`await`**: A "suspension point." It tells the code to stop and wait for an `async` function to return a result before moving to the next line.
- **`Task`**: The bridge that allows you to run asynchronous code from a synchronous environment (like a button tap in a View Controller).

```
None
func fetchData() async -> String {
    try? await Task.sleep(nanoseconds: 1_000_000_000) //
Simulates 1s delay
    return "Data Received"
}

// Usage
Task {
    let result = await fetchData()
    print(result)
}
```

Fixed Parallelism: `async let`

Used when you have a **fixed, known number of tasks** that can run at the same time. It is simple and readable.

Example:

Swift

```
None
func loadUserDashboard() async {
    async let profile = fetchProfile()
    async let notifications = fetchNotifications()

    // Both start at once; we wait for both here
    let (user, notes) = await (profile, notifications)
}
```

Dynamic Parallelism: `TaskGroup`

Used when the number of tasks is **unknown or dynamic** (e.g., a list of 10 or 50 URLs). It allows you to add tasks in a loop.

Example:

Swift

```
None
func downloadAll(urls: [URL]) async {
    await withTaskGroup(of: UIImage.self) { group in
        for url in urls {
            group.addTask { await downloadImage(url) }
        }

        for await image in group {
            // Process each image as it finishes
        }
    }
}
```

Data Safety: `actor` vs `DispatchQueue`

When multiple background tasks try to change the same variable (like an array of images), it causes a **Data Race**.

- `actor`: The modern solution. It automatically ensures only one task can access its data at a time. No manual locking is required.
- `DispatchQueue`: The "classic" way. Requires you to manually wrap code in a queue to prevent crashes.

Example (Modern Actor):

Swift

```
None
actor ImageVault {
    var list: [UIImage] = []
    func save(_ img: UIImage) { list.append(img) }
}

// Usage inside a Task Group
await vault.save(downloadedImage) // Safe and synchronized
```

Thread Management: Main vs. Background

- **Main Thread**: For UI only (labels, buttons, scrolling).
- **Background**: For heavy tasks (API calls, image processing).
- `@MainActor`: A simple attribute that forces a function or class to always run on the Main Thread, preventing UI-related crashes.

Example:

Swift

```
None
@MainActor
func updateUI(with image: UIImage) {
    self.imageView.image = image // It will be on Main Thread
}
```

Cooperative Task Cancellation

In Swift, stopping a task isn't like pulling a plug; it's a polite request. If a user navigates away from a screen, you should stop the download to save battery and data.

- **The Concept:** When a parent task is cancelled, all child tasks are marked as "cancelled," but they don't stop automatically. You must check for it.
- **The Check:** Use `Task.checkCancellation()` (which throws an error) or `Task.isCancelled` (a boolean).

Example:

Swift

```
None
group.addTask {
    // Check if the user closed the screen before starting the
heavy work
    try Task.checkCancellation()
    return try await self.downloadSingle(url: url)
}
```

Task Priority

Not all background tasks are equal. Downloading a profile picture for the current screen is more important than pre-loading a video for three screens away.

- **The Concept:** You can tell the system how urgent a task is.
- **The Levels:**
  - `.userInitiated`: High priority (User is waiting).
  - `.medium` / `.default`: Standard tasks.
  - `.utility`: Low priority (Downloading data in the background).
  - `.background`: Lowest priority (Cleanup, indexing).

Example:

Swift

```
None
Task(priority: .userInitiated) {
    let image = await manager.downloadFeaturedImages()
}
```

Sendable Types

This is the "Rules of the Road" for concurrency. As you move data between an `actor` and a `TaskGroup`, Swift needs to know that data is safe to "send" across threads.

- **The Concept:** A type is `Sendable` if it cannot cause a data race when shared.
- **What is Sendable?** * Value types (Structs, Enums, Strings, Ints).
    - Actors (because they protect their own state).
    - Classes that are "Final" and have only "let" (constant) properties.
- **Why it matters:** If you try to pass a regular, mutable `class` between threads, the compiler will warn you that it's "Non-Sendable" and dangerous.

qImplicit vs. Explicit Sendability

Swift is very smart about which types are safe to move between threads (`Sendable`).

- **Implicitly Sendable:** Most `structs` and `enums` are automatically Sendable if their properties are also Sendable. You don't have to write the code yourself.
- **Explicitly NOT Sendable:** If you have a type that is dangerous (like a raw file pointer), you can explicitly tell Swift to **forbid** it from being sent across threads.

Example:

Swift

```
None
// A simple struct is implicitly Sendable
```

```
struct UserProfile {
    let name: String // String is Sendable, so UserProfile is too
}

// Manually marking a type as NOT safe for concurrency
@available(*, unavailable)
extension MyDangerousClass: Sendable {}
```

Continuations

You might encounter old SDKs that use completion handlers (closures) instead of `async/await`.

- **The Concept:** `withCheckedContinuation` allows you to wrap an old callback-style function so it works with `await`.

Example:

Swift

```
None
// Wrapping an old callback into a modern async function
func fetchOldWay() async -> String {
    await withCheckedContinuation { continuation in
        oldAPI.getData { result in
            continuation.resume(returning: result)
        }
    }
```

```
}
```

Task Reentrancy (The Actor "Trap")

This is the most common mistake with Actors.

- **The Concept:** When an Actor is `awaiting` something (like a download), it **unlocks itself** so other tasks can run.
- **The Risk:** If you aren't careful, the state of your Actor might change while you are waiting for a download to finish.

Example of the "Trap":

Swift

```
None
actor ImageStore {
    var count = 0
    func addImage() async {
        count += 1
        await someLongDownload() // The actor is now "open" for
others!
        print(count) // This might NOT be what you expect if
another task ran
    }
}
```

Actor Reentrancy (The "Gap" in Safety)

While actors protect your data, they have a specific behavior called **Reentrancy**. When an actor hits an `await` point, it **suspends** and allows other tasks to run on that same actor.

- **The Risk:** If you change a variable, then `await` a long task, another task might change that same variable before you "wake up."

- **The Rule:** Code between two `await` points is **atomic** (cannot be interrupted). But across an `await` point, the state of the actor can change.

Example of the Reentrancy "Trap":

Swift

```
None
actor ImageStore {
    var totalDownloaded = 0

    func downloadAndIncrement() async {
        totalDownloaded += 1
        // --- SUSPENSION POINT ---
        await someLongNetworkCall()
        // By the time we reach here, another task could have
        // changed 'totalDownloaded'!
        print(totalDownloaded)
    }
}
```

Asynchronous Sequences (`AsyncSequence`)

Sometimes you don't want to wait for an entire array to download before you start working. An `AsyncSequence` allows you to receive and process items **one by one** as they become available.

- **The Concept:** Instead of a standard `for-in` loop, you use `for await-in`.
- **Suspension:** The loop pauses at the start of every iteration until the next element is ready. It doesn't block the thread while waiting.

Example:

Swift

```
None
// Reading a file line-by-line asynchronously
let handle = FileHandle.standardInput
```

```
for try await line in handle.bytes.lines {
    // This line runs every time a new "line" arrives
    print("Received: \(line)")
```

Async Streams (`AsyncStream`)

Your current demo downloads a bunch of images and returns them all at once. But what if you wanted to update the UI **every time one image finished**?

- **The Concept:** An `AsyncStream` is like a pipe. You can "push" values into it over time, and the caller can "loop" over the pipe.
- **Analogy:** It's like a Twitter feed that keeps loading new tweets as you scroll, rather than a single static web page.

Structured vs. Unstructured Concurrency

You are already using both, but knowing the *vocabulary* is what impresses interviewers:

- **Structured:** `async let` and `TaskGroup`. The parent task "owns" the children. If the parent is cancelled, the children are cancelled. This is **safe**.
- **Unstructured:** `Task { ... }`. You are starting a "rogue" task that isn't tied to the current function. It’s powerful but harder to manage (you have to manually cancel it).

URL and URLRequest

Before you can download anything, you need to tell Swift **where** to go.

- **URL:** A data type that represents the location of a resource (like an image on a server).
- **Validation:** Creating a URL can fail if the string is badly formatted, so we often use `if let` or `guard`.

Example:

Swift

```
None
guard let url = URL(string: "https://api.example.com/data") else
{
    print("Invalid URL")
    return
}
```

URLSession (The Network Engine)

`URLSession` is the object responsible for sending and receiving network requests. In modern Swift, it provides an `async` method that replaces the old "Completion Handlers."

- **`data(from:)`**: This is an asynchronous function. It pauses (suspends) your code, frees up the thread for the UI, and resumes only when the data has finished downloading.
- **The Tuple Return:** It returns two things at once:
    1. **`Data`**: The actual bytes downloaded (the image, JSON, etc.).
    2. **`URLResponse`**: Information about the delivery (e.g., "Status Code 200" means Success).

Example:

Swift

```
None
// The "await" tells the system: "Go download this, I'll wait
here."
let (data, response) = try await URLSession.shared.data(from:
url)

// Always check if the server actually said 'OK' (status 200)
if let httpResponse = response as? HTTPURLResponse,
httpResponse.statusCode == 200 {
    // Process your data here
}
```

Swift UIkit:

File structure:

1. The "Logic" Files

`ViewController.swift`

This is your primary workspace. In UIKit (the framework behind Storyboards), every screen is managed by a **View Controller**.

- **The Job:** It acts as the "brain" for one screen. It handles button taps, fetches data, and tells the UI when to update.
- **The Lifecycle:** You'll see `viewDidLoad()`—this is the most important method. It's where you do your initial setup (like setting a background color or starting a timer) as soon as the screen is "born."

`AppDelegate.swift`

This is the "Grand Manager" of the entire application.

- **The Job:** It handles app-level events. For example, what should happen when the app finishes launching? What if a push notification arrives?
- **The Scope:** It doesn't care about what's happening on a specific screen; it cares about the life of the process itself.

`SceneDelegate.swift`

This is a newer addition (since iOS 13) that handles **Window Management**.

- **The Job:** On iPads, you can have two windows of the same app open side-by-side. The `SceneDelegate` manages these "Scenes."
- **The Connection:** It tells the app which Storyboard to load first and attaches it to a "Window" so the user can actually see it.

2. The "Visual" & "Config" Files

`Main.storyboard`

This is your **Visual Canvas**.

- **The Job:** You drag buttons, labels, and images from the Library (`Cmd + Shift + L`) onto this canvas.
- **The Wiring:** You connect these visual elements to your `ViewController.swift` using **IBOutlets** (to change properties) and **IBActions** (to respond to clicks).

`Assets.xcassets`

The "Storage Closet."

- **The Job:** This is where you put your images, app icons, and custom color sets.
- **Why use it?** It handles "Scale Factors" for you. You can provide a standard image, a high-res version (@2x), and a super high-res version (@3x), and the iPhone will automatically pick the right one.

`LaunchScreen.storyboard`

The "Waiting Room."

- **The Job:** This is a static screen that shows up for a split second while your app is loading into memory.
- **Restriction:** You cannot put code here. It's just a simple layout to make the app feel fast and responsive.

`Info` (formerly Info.plist)

The "ID Card."

- **The Job:** This contains the app's metadata.
- **Permissions:** If your app needs to use the Camera or Location, you *must* add a description here explaining why. If you don't, the app will crash the moment it tries to access those features.

---

3. How they all work together

1. **System:** User taps the app icon.
2. **AppDelegate:** "Okay, the app is starting up!"
3. **SceneDelegate:** "I'm creating a new window. I'll load `Main.storyboard`."
4. **Storyboard:** "I'm the visual layout. I'm connected to `ViewController.swift`."
5. **ViewController:** `viewDidLoad()` runs, your code executes, and the user sees your screen.

Attributes:

1. @IBOutlet (The "Property")

`@IBOutlet` stands for **Interface Builder Outlet**.

- **Purpose:** Use this when you want the **code to talk to the UI**.
- **Example:** You want to change a Label's text or hide a Button.
- **Analogy:** It’s like a **remote control** pointing at the TV.

Swift

```
None
@IBOutlet weak var myLabel: UILabel!
```

- **`weak`**: This is a memory management keyword. Because the "View" already owns the label, the "Controller" holds a "weak" reference to avoid a memory leak (a "Retain Cycle").

- `!`: This is an **Implicitly Unwrapped Optional**. It tells Swift: "This might be nil for a second, but I promise by the time I use it, the Storyboard will have connected it."

That `@IB` prefix is the "secret sauce" that lets the **Storyboard** (a visual file) talk to your **Swift code.** In professional terms, these are called **Attributes**. They tell the Swift compiler: "Hey, keep this variable or function available so the Interface Builder (IB) can see it."

1. @IBOutlet (The "Property")

`@IBOutlet` stands for **Interface Builder Outlet**.

- **Purpose:** Use this when you want the **code to talk to the UI**.
- **Example:** You want to change a Label's text or hide a Button.
- **Analogy:** It's like a **remote control** pointing at the TV.

```
None
@IBOutlet weak var myLabel: UILabel!
```

- `weak`: This is a memory management keyword. Because the "View" already owns the label, the "Controller" holds a "weak" reference to avoid a memory leak (a "Retain Cycle").
- `!`: This is an **Implicitly Unwrapped Optional**. It tells Swift: "This might be nil for a second, but I promise by the time I use it, the Storyboard will have connected it."

---

2. @IBAction (The "Event")

`@IBAction` stands for **Interface Builder Action**.

- **Purpose:** Use this when you want the **UI to talk to the code**.
- **Example:** The user taps a button, and you want to start a download.
- **Analogy:** It's like the **light switch** on the wall. When you flip it, the electricity (code) flows.

```
None
@IBAction func downloadButtonTapped(_ sender: UIButton) {
    // Your code goes here!
    print("User wants to download images.")
}
```

- `_ sender: UIButton`: This tells you *which* button was pressed. If you have five buttons connected to the same function, the `sender` is the one that actually got clicked.